

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent	12401599
Kind Code	B2
Date of Patent	August 26, 2025
Inventor(s)	Lambeth; W. Andrew et al.

Inspecting operations of a machine to detect elephant flows

Abstract

Some embodiments provide a system that detects whether a data flow is an elephant flow; and if so, the system treats it differently than a mouse flow. The system of some embodiments detects an elephant flow by examining, among other items, the operations of a machine. In detecting, the system identifies an initiation of a new data flow associated with the machine. The new data flow can be an outbound data flow or an inbound data flow. The system then determines, based on the amount of data being sent or received, if the data flow is an elephant flow. The system of some embodiments identifies the initiation of a new data flow by intercepting a socket call or request to transfer a file.

Inventors: Lambeth; W. Andrew (San Francisco, CA), Patil; Amit Vasant (Pune, IN), Dabak; Prasad Sharad (Pune, IN), Gunda; Laxmikant Vithal (San Jose, CA), Dhanasekar; Vasantha Kumar (Tamil Nadu, IN), Pettit; Justin (Los Altos Hills, CA)

Applicant: VMware LLC (Palo Alto, CA)

Family ID: 1000008778226

Assignee: VMware LLC (Palo Alto, CA)

Appl. No.: 18/226774

Filed: July 27, 2023

Prior Publication Data

Document Identifier	Publication Date
US 20230379260 A1	Nov. 23, 2023

Related U.S. Application Data

continuation parent-doc US 18088601 20221225 US 11811669 child-doc US 18226774
continuation parent-doc US 15972232 20180507 US 11539630 20221227 child-doc US 18088601

continuation parent-doc US 14502102 20140930 US 9967199 20180508 child-doc US 15972232
us-provisional-application US 62010944 20140611
us-provisional-application US 61973255 20140331
us-provisional-application US 61913899 20131209

Publication Classification

Int. Cl.: **H04L47/2441** (20220101); **H04L43/026** (20220101)

U.S. Cl.:

CPC **H04L47/2441** (20130101); **H04L43/026** (20130101);

Field of Classification Search

CPC: H04L (12/26); H04L (47/2441); H04L (43/026)

USPC: 709/224

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
5224100	12/1992	Lee et al.	N/A	N/A
5245609	12/1992	Ofek et al.	N/A	N/A
5265092	12/1992	Soloway et al.	N/A	N/A
5504921	12/1995	Dev et al.	N/A	N/A
5550816	12/1995	Hardwick et al.	N/A	N/A
5668810	12/1996	Cannella	N/A	N/A
5729685	12/1997	Chatwani et al.	N/A	N/A
5751967	12/1997	Raab et al.	N/A	N/A
5781534	12/1997	Perlman et al.	N/A	N/A
6044142	12/1999	Hammarstrom	379/201.01	H04M 3/51
6104699	12/1999	Holender et al.	N/A	N/A
6104700	12/1999	Haddock et al.	N/A	N/A
6141738	12/1999	Munter et al.	N/A	N/A
6219699	12/2000	McCloghrie et al.	N/A	N/A
6430160	12/2001	Smith et al.	N/A	N/A
6456624	12/2001	Eccles et al.	N/A	N/A
6512745	12/2002	Abe et al.	N/A	N/A
6539432	12/2002	Taguchi et al.	N/A	N/A
6658002	12/2002	Ross et al.	N/A	N/A
6680934	12/2003	Cain	N/A	N/A
6721334	12/2003	Ketcham	N/A	N/A
6785843	12/2003	McRae et al.	N/A	N/A
6941487	12/2004	Balakrishnan et al.	N/A	N/A
6963585	12/2004	Pennec et al.	N/A	N/A
6999454	12/2005	Crump	N/A	N/A
7012919	12/2005	So et al.	N/A	N/A

7042838	12/2005	Shand	370/221	H04L 45/00
7079544	12/2005	Wakayama et al.	N/A	N/A
7149817	12/2005	Pettey	N/A	N/A
7149819	12/2005	Pettey	N/A	N/A
7197572	12/2006	Matters et al.	N/A	N/A
7200144	12/2006	Terrell et al.	N/A	N/A
7209439	12/2006	Rawlins et al.	N/A	N/A
7283473	12/2006	Arndt et al.	N/A	N/A
7342916	12/2007	Das et al.	N/A	N/A
7362752	12/2007	Kastenholz	N/A	N/A
7370120	12/2007	Kirsch et al.	N/A	N/A
7391771	12/2007	Orava et al.	N/A	N/A
7443796	12/2007	Albert	709/239	H04L 67/1001
7450598	12/2007	Chen et al.	N/A	N/A
7463579	12/2007	Lapuh et al.	N/A	N/A
7478173	12/2008	Delco	N/A	N/A
7483370	12/2008	Dayal et al.	N/A	N/A
7529897	12/2008	Waldspurger	714/13	G06F 11/1482
7533176	12/2008	Freimuth et al.	N/A	N/A
7555002	12/2008	Arndt et al.	N/A	N/A
7606260	12/2008	Oguchi et al.	N/A	N/A
7627692	12/2008	Pessi	N/A	N/A
7633955	12/2008	Saraiya et al.	N/A	N/A
7634622	12/2008	Musoll et al.	N/A	N/A
7640353	12/2008	Shen et al.	N/A	N/A
7643488	12/2009	Khanna et al.	N/A	N/A
7649851	12/2009	Takashige et al.	N/A	N/A
7706266	12/2009	Plamondon	N/A	N/A
7710874	12/2009	Balakrishnan et al.	N/A	N/A
7760735	12/2009	Chen et al.	N/A	N/A
7764599	12/2009	Doi et al.	N/A	N/A
7792987	12/2009	Vohra et al.	N/A	N/A
7802000	12/2009	Huang et al.	N/A	N/A
7808919	12/2009	Nadeau et al.	N/A	N/A
7808929	12/2009	Wong et al.	N/A	N/A
7818452	12/2009	Matthews et al.	N/A	N/A
7826482	12/2009	Minei et al.	N/A	N/A
7839847	12/2009	Nadeau et al.	N/A	N/A
7885276	12/2010	Lin	N/A	N/A
7936770	12/2010	Frattura et al.	N/A	N/A
7937438	12/2010	Miller et al.	N/A	N/A
7937492	12/2010	Kompella et al.	N/A	N/A
7940763	12/2010	Kastenholz	N/A	N/A
7948986	12/2010	Ghosh et al.	N/A	N/A
7953865	12/2010	Miller et al.	N/A	N/A
7991859	12/2010	Miller et al.	N/A	N/A
7995483	12/2010	Bayar et al.	N/A	N/A
8004900	12/2010	Dutta et al.	N/A	N/A

8004990	12/2010	Callon	N/A	N/A
8027354	12/2010	Portolani et al.	N/A	N/A
8031606	12/2010	Memon et al.	N/A	N/A
8031633	12/2010	Bueno et al.	N/A	N/A
8046456	12/2010	Miller et al.	N/A	N/A
8054832	12/2010	Shukla et al.	N/A	N/A
8055789	12/2010	Richardson et al.	N/A	N/A
8060875	12/2010	Lambeth	N/A	N/A
8131852	12/2011	Miller et al.	N/A	N/A
8149737	12/2011	Metke et al.	N/A	N/A
8155028	12/2011	Abu-Hamdeh et al.	N/A	N/A
8161270	12/2011	Parker et al.	N/A	N/A
8166201	12/2011	Richardson et al.	N/A	N/A
8199750	12/2011	Schultz et al.	N/A	N/A
8223668	12/2011	Allan et al.	N/A	N/A
8224931	12/2011	Brandwine et al.	N/A	N/A
8224971	12/2011	Miller et al.	N/A	N/A
8265075	12/2011	Pandey	N/A	N/A
8281067	12/2011	Stolowitz	N/A	N/A
8312129	12/2011	Miller et al.	N/A	N/A
8339959	12/2011	Moisand et al.	N/A	N/A
8339994	12/2011	Gnanasekaran et al.	N/A	N/A
8345558	12/2012	Nicholson et al.	N/A	N/A
8351418	12/2012	Zhao et al.	N/A	N/A
8355328	12/2012	Matthews et al.	N/A	N/A
8456984	12/2012	Ranganathan et al.	N/A	N/A
8504718	12/2012	Wang et al.	N/A	N/A
8571031	12/2012	Davies et al.	N/A	N/A
8611351	12/2012	Gooch et al.	N/A	N/A
8612627	12/2012	Brandwine	N/A	N/A
8619731	12/2012	Montemurro et al.	N/A	N/A
8625594	12/2013	Safrai et al.	N/A	N/A
8625603	12/2013	Ramakrishnan et al.	N/A	N/A
8625616	12/2013	Vobbilisetty et al.	N/A	N/A
8644188	12/2013	Brandwine et al.	N/A	N/A
8677451	12/2013	Bhimaraju	726/8	G06F 21/41
8762501	12/2013	Kempf et al.	N/A	N/A
8819259	12/2013	Zuckerman et al.	N/A	N/A
8838743	12/2013	Lewites et al.	N/A	N/A
8976814	12/2014	Dipasquale	N/A	N/A
9032095	12/2014	Traina et al.	N/A	N/A
9548924	12/2016	Pettit et al.	N/A	N/A
9548942	12/2016	Deluca et al.	N/A	N/A
9762507	12/2016	Gandham et al.	N/A	N/A
9838276	12/2016	Pettit et al.	N/A	N/A
9967199	12/2017	Lambeth et al.	N/A	N/A
10158538	12/2017	Pettit et al.	N/A	N/A
10193771	12/2018	Koponen et al.	N/A	N/A
10498638	12/2018	Jackson et al.	N/A	N/A

10516612	12/2018	Dharmapurikar	N/A	H03M 13/11
10666530	12/2019	Koponen et al.	N/A	N/A
11095536	12/2020	Koponen et al.	N/A	N/A
11539630	12/2021	Lambeth et al.	N/A	N/A
11811669	12/2022	Lambeth	N/A	H04L 47/2441
2001/0043614	12/2000	Viswanadham et al.	N/A	N/A
2002/0062422	12/2001	Butterworth et al.	N/A	N/A
2002/0093952	12/2001	Gonda	N/A	N/A
2002/0194369	12/2001	Rawlins et al.	N/A	N/A
2003/0028451	12/2002	Ananian	705/26.42	G06Q 30/0615
2003/0041170	12/2002	Suzuki	N/A	N/A
2003/0058850	12/2002	Rangarajan et al.	N/A	N/A
2003/0063556	12/2002	Hernandez	N/A	N/A
2003/0069972	12/2002	Yoshimura et al.	N/A	N/A
2003/0093341	12/2002	Millard et al.	N/A	N/A
2003/0140165	12/2002	Chiu et al.	N/A	N/A
2003/0191841	12/2002	DeFerranti et al.	N/A	N/A
2004/0073659	12/2003	Rajsic et al.	N/A	N/A
2004/0098505	12/2003	Clemmensen	N/A	N/A
2004/0186914	12/2003	Shimada	N/A	N/A
2004/0264472	12/2003	Oliver et al.	N/A	N/A
2004/0267866	12/2003	Carollo et al.	N/A	N/A
2004/0267897	12/2003	Hill et al.	N/A	N/A
2005/0018669	12/2004	Arndt et al.	N/A	N/A
2005/0027881	12/2004	Figueira et al.	N/A	N/A
2005/0053079	12/2004	Havala	N/A	N/A
2005/0083953	12/2004	May	N/A	N/A
2005/0102401	12/2004	Patrick	709/225	G06F 21/6218
2005/0111445	12/2004	Wybenga et al.	N/A	N/A
2005/0120160	12/2004	Plouffe et al.	N/A	N/A
2005/0132044	12/2004	Guingo et al.	N/A	N/A
2005/0182853	12/2004	Lewites et al.	N/A	N/A
2005/0220096	12/2004	Friskney et al.	N/A	N/A
2005/0232230	12/2004	Nagami et al.	N/A	N/A
2006/0002370	12/2005	Rabie et al.	N/A	N/A
2006/0026225	12/2005	Canali et al.	N/A	N/A
2006/0028999	12/2005	Takobashvili et al.	N/A	N/A
2006/0029056	12/2005	Perera et al.	N/A	N/A
2006/0037075	12/2005	Frattura et al.	N/A	N/A
2006/0104286	12/2005	Cheriton	N/A	N/A
2006/0140118	12/2005	Alicherry et al.	N/A	N/A
2006/0174087	12/2005	Hashimoto et al.	N/A	N/A
2006/0187908	12/2005	Shimozono et al.	N/A	N/A
2006/0193266	12/2005	Siddha et al.	N/A	N/A
2006/0206655	12/2005	Chappell et al.	N/A	N/A
2006/0221961	12/2005	Basso et al.	N/A	N/A

2006/0246900	12/2005	Zheng	N/A	N/A
2006/0262778	12/2005	Haumont et al.	N/A	N/A
2006/0282895	12/2005	Rentzis et al.	N/A	N/A
2006/0291388	12/2005	Amdahl et al.	N/A	N/A
2007/0050763	12/2006	Kagan et al.	N/A	N/A
2007/0055789	12/2006	Claise et al.	N/A	N/A
2007/0064673	12/2006	Bhandaru et al.	N/A	N/A
2007/0156919	12/2006	Potti et al.	N/A	N/A
2007/0195797	12/2006	Patel	370/469	H04L 47/193
2007/0206086	12/2006	Baron	348/E7.081	H04N 7/147
2007/0258382	12/2006	Foll et al.	N/A	N/A
2007/0260721	12/2006	Bose et al.	N/A	N/A
2007/0283412	12/2006	Lie et al.	N/A	N/A
2007/0286185	12/2006	Eriksson et al.	N/A	N/A
2007/0297428	12/2006	Bose et al.	N/A	N/A
2008/0002579	12/2007	Lindholm et al.	N/A	N/A
2008/0002683	12/2007	Droux et al.	N/A	N/A
2008/0049614	12/2007	Briscoe et al.	N/A	N/A
2008/0049621	12/2007	McGuire et al.	N/A	N/A
2008/0049786	12/2007	Ram et al.	N/A	N/A
2008/0059556	12/2007	Greenspan et al.	N/A	N/A
2008/0071900	12/2007	Hecker et al.	N/A	N/A
2008/0086726	12/2007	Griffith et al.	N/A	N/A
2008/0159301	12/2007	Heer	N/A	N/A
2008/0240095	12/2007	Basturk	N/A	N/A
2009/0006607	12/2008	Bu et al.	N/A	N/A
2009/0010254	12/2008	Shimada	N/A	N/A
2009/0046581	12/2008	Eswaran et al.	N/A	N/A
2009/0150527	12/2008	Tripathi et al.	N/A	N/A
2009/0241117	12/2008	Dasgupta	718/101	G06F 9/5038
2009/0282485	12/2008	Bennett	707/999.003	H04L 63/145
2009/0292858	12/2008	Lambeth et al.	N/A	N/A
2010/0061242	12/2009	Sindhu	370/235	H04L 49/30
2010/0122343	12/2009	Ghosh	726/23	G06F 9/45545
2010/0128623	12/2009	Dunn et al.	N/A	N/A
2010/0131636	12/2009	Suri et al.	N/A	N/A
2010/0157942	12/2009	An et al.	N/A	N/A
2010/0214949	12/2009	Smith et al.	N/A	N/A
2010/0232435	12/2009	Jabr et al.	N/A	N/A
2010/0254385	12/2009	Sharma et al.	N/A	N/A
2010/0257263	12/2009	Casado et al.	N/A	N/A
2010/0275199	12/2009	Smith et al.	N/A	N/A
2010/0306408	12/2009	Greenberg et al.	N/A	N/A
2011/0022695	12/2010	Dalal et al.	N/A	N/A

2011/0060861	12/2010	Warren	711/E12.001	G06F 12/0246
2011/0075664	12/2010	Lambeth et al.	N/A	N/A
2011/0085461	12/2010	Liu	370/252	H04L 43/12
2011/0085557	12/2010	Gnanasekaran et al.	N/A	N/A
2011/0085559	12/2010	Chung et al.	N/A	N/A
2011/0085563	12/2010	Kotha et al.	N/A	N/A
2011/0128959	12/2010	Bando et al.	N/A	N/A
2011/0164503	12/2010	Yong et al.	N/A	N/A
2011/0185292	12/2010	Chawla	709/227	G06F 9/5077
2011/0185355	12/2010	Chawla	718/1	G06F 9/45558
2011/0194567	12/2010	Shen	N/A	N/A
2011/0202920	12/2010	Takase	N/A	N/A
2011/0249970	12/2010	Eddleston et al.	N/A	N/A
2011/0261825	12/2010	Ichino	N/A	N/A
2011/0299413	12/2010	Chatwani et al.	N/A	N/A
2011/0299534	12/2010	Koganti et al.	N/A	N/A
2011/0299537	12/2010	Saraiya et al.	N/A	N/A
2011/0305167	12/2010	Koide	N/A	N/A
2011/0317559	12/2010	Kern et al.	N/A	N/A
2011/0317696	12/2010	Aldrin et al.	N/A	N/A
2012/0054367	12/2011	Ramakrishnan et al.	N/A	N/A
2012/0079478	12/2011	Galles et al.	N/A	N/A
2012/0131222	12/2011	Curtis	709/235	H04L 47/10
2012/0159454	12/2011	Barham et al.	N/A	N/A
2012/0182992	12/2011	Cowart et al.	N/A	N/A
2012/0233349	12/2011	Aybay	709/234	H04L 47/2441
2012/0243539	12/2011	Keesara	N/A	N/A
2012/0287791	12/2011	Xi et al.	N/A	N/A
2013/0024579	12/2012	Zhang et al.	N/A	N/A
2013/0054761	12/2012	Kempf et al.	N/A	N/A
2013/0058346	12/2012	Sridharan et al.	N/A	N/A
2013/0064088	12/2012	Yu et al.	N/A	N/A
2013/0067067	12/2012	Miri et al.	N/A	N/A
2013/0163427	12/2012	Beliveau et al.	N/A	N/A
2013/0163475	12/2012	Beliveau et al.	N/A	N/A
2013/0286846	12/2012	Atlas et al.	N/A	N/A
2013/0287026	12/2012	Davie	N/A	N/A
2013/0322248	12/2012	Guo	N/A	N/A
2013/0332602	12/2012	Nakil et al.	N/A	N/A
2013/0339544	12/2012	Mithyantha	N/A	N/A
2014/0010235	12/2013	Ueno	370/392	H04L 43/026
2014/0019639	12/2013	Ueno	N/A	N/A
2014/0029451	12/2013	Nguyen	N/A	N/A
2014/0108738	12/2013	Kim et al.	N/A	N/A
2014/0115578	12/2013	Cooper et al.	N/A	N/A

2014/0119203	12/2013	Sundaram et al.	N/A	N/A
2014/0173018	12/2013	Westphal et al.	N/A	N/A
2014/0195666	12/2013	Dumitriu et al.	N/A	N/A
2014/0233421	12/2013	Matthews	370/253	H04L 49/35
2014/0281030	12/2013	Cui et al.	N/A	N/A
2014/0297597	12/2013	Matsubara	707/681	G06F 11/1458
2014/0359134	12/2013	Yoshida	709/226	H04L 47/11
2014/0369204	12/2013	Anand	370/235.1	H04L 67/1004
2014/0372616	12/2013	Arisoylu et al.	N/A	N/A
2015/0009830	12/2014	Bisht	370/236	H04L 41/0895
2015/0016255	12/2014	Bisht et al.	N/A	N/A
2015/0016265	12/2014	Ahmadi	N/A	N/A
2015/0026794	12/2014	Zuk	726/13	H04L 69/22
2015/0071072	12/2014	Ratzin	370/235	G06F 9/544
2015/0071076	12/2014	Izhak-Ratzin	370/236	H04L 47/2408
2015/0106804	12/2014	Chandrashekhar et al.	N/A	N/A
2015/0120959	12/2014	Bennett et al.	N/A	N/A
2015/0124825	12/2014	Dharmapurikar	370/392	H04L 45/7453
2015/0163117	12/2014	Lambeth et al.	N/A	N/A
2015/0163142	12/2014	Pettit et al.	N/A	N/A
2015/0163144	12/2014	Koponen et al.	N/A	N/A
2015/0163145	12/2014	Pettit et al.	N/A	N/A
2015/0163146	12/2014	Zhang et al.	N/A	N/A
2015/0172075	12/2014	DeCusatis et al.	N/A	N/A
2015/0180769	12/2014	Wang et al.	N/A	N/A
2015/0237097	12/2014	Devireddy	709/203	H04L 65/80
2015/0281102	12/2014	Lee et al.	N/A	N/A
2015/0341247	12/2014	Curtis et al.	N/A	N/A
2016/0094643	12/2015	Jain et al.	N/A	N/A
2016/0105333	12/2015	Lenglet et al.	N/A	N/A
2016/0127227	12/2015	Lee et al.	N/A	N/A
2016/0134511	12/2015	Wang et al.	N/A	N/A
2016/0156591	12/2015	Zhou et al.	N/A	N/A
2016/0182454	12/2015	Phonsa et al.	N/A	N/A
2017/0118090	12/2016	Pettit et al.	N/A	N/A
2018/0048571	12/2017	Dharmapurikar	N/A	H03M 13/11
2018/0331961	12/2017	Andrew et al.	N/A	N/A
2019/0140924	12/2018	Koponen et al.	N/A	N/A
2020/0351181	12/2019	Koponen et al.	N/A	N/A
2021/0377134	12/2020	Koponen et al.	N/A	N/A
2023/0128660	12/2022	Lambeth et al.	N/A	N/A

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
1154601	12/2000	EP	N/A
2002141905	12/2001	JP	N/A
2003069609	12/2002	JP	N/A
2003124976	12/2002	JP	N/A
2003318949	12/2002	JP	N/A
9506989	12/1994	WO	N/A
2004047377	12/2003	WO	N/A
2012126488	12/2011	WO	N/A
2013184846	12/2012	WO	N/A

OTHER PUBLICATIONS

Mahout: Low-Overhead Datacenter Traffic Management using End-Host-Based Elephant Detection. Andrew R. Curtis, Wonho Kim and Praveen Yalagandula pp. 1629-1637, IEEE inforcome 2011. cited by examiner

Anwer, Muhammad Bilal, et al., "Building a Fast, Virtualized Data Plane with Programmable Hardware," Aug. 17, 2009, 8 pages, VISA'09, ACM, Barcelona, Spain. cited by applicant

Casado, Martin, et al. "Ethane: Taking Control of the Enterprise," SIGCOMM'07, Aug. 27-31, 2007, 12 pages, ACM, Kyoto, Japan. cited by applicant

Foster, Nate, et al. "Frenetic: A Network Programming Language," ICFP '11, Sep. 19-21, 2011, 13 pages, Tokyo, Japan. cited by applicant

Gude, Natasha, et al., "NOX: Towards an Operating System for Networks," ACM SIGCOMM Computer Communication Review, Jul. 2008, 6 pages, vol. 38, No. 3, ACM. cited by applicant

Mckeown, Nick, et al., "OpenFlow: Enabling Innovation in Campus Networks," ACM SIGCOMM Computer Communication Review, Apr. 2008, 6 pages, vol. 38, No. 2, ACM. cited by applicant

Pfaff, Ben., et al., "Extending Networking into the Virtualization Layer," Proc. of HotNets, Oct. 2009, 6 pages. cited by applicant

Phan, Doantam, et al., "Visual Analysis of Network Flow Data with Timelines and Event Plots, " VizSEC 2007, Month Unknown 2007, 16 pages. cited by applicant

Tavakoli, Arsalan, et al., "Applying NOX to the Datacenter," Proc. HotNets, Month Unknown 2009, 6 pages. cited by applicant

Primary Examiner: Hussain; Tauqir

Attorney, Agent or Firm: Quarles & Brady LLP

Background/Summary

CLAIM OF BENEFIT TO PRIOR APPLICATIONS (1) This application is a continuation application of U.S. patent application Ser. No. 18/088,601, filed Dec. 25, 2022, now published as U.S. Patent Publication 2023/0128660. U.S. patent application Ser. No. 18/088,601 is a continuation application of U.S. patent application Ser. No. 15/972,232, filed May 7, 2018, now issued as U.S. Pat. No. 11,539,630. U.S. patent application Ser. No. 15/972,232 is a continuation application of U.S. patent application Ser. No. 14/502,102, filed Sep. 30, 2014, now issued as U.S. Pat. No. 9,967,199. U.S. patent application Ser. No. 14/502,102 claims the benefit of U.S. Provisional Patent Application 61/913,899, filed Dec. 9, 2013, U.S. Provisional Patent Application 61/973,255, filed Mar. 31, 2014, and U.S. Provisional Patent Application 62/010,944, filed Jun. 11, 2014. U.S. patent application Ser. No. 18/088,601, now published as U.S. Patent Publication

2023/0128660, U.S. patent application Ser. No. 15/972,232, now issued as U.S. Pat. No. 11,539,630, U.S. patent application Ser. No. 14/502,102, now issued as U.S. Pat. No. 9,967,199, and U.S. Provisional Patent Applications 61/913,899, 61/973,255, and 62/010,944 are incorporated herein by reference.

BACKGROUND

(1) Today, a datacenter may process different types of flows, including elephant flows and mouse flows. An elephant flow represents a long-lived flow or a continuous traffic flow that is typically associated with high volume connection. Different from an elephant flow, a mouse flow represents a short-lived flow. Mice are often associated with bursty, latency-sensitive applications, whereas elephants tend to be associated with large data transfers in which throughput is far more important than latency.

(2) A problem with elephant flows is that they tend to fill network buffers end-to-end, and this introduces non-trivial queuing delay to anything that shares these buffers. For instance, a forwarding element may be responsible for managing several queues to forward packets, and several packets belonging to a mouse flow may be stuck in the same queue behind a group of other packets belonging to an elephant flow. In a network of elephants and mice, this means that the more latency-sensitive mice are being affected. Another problem is that mice are generally very bursty, so adaptive routing techniques are not effective with them.

BRIEF SUMMARY

(3) Some embodiments provide a system that detects whether a data flow is an elephant flow; and if so, the system treats it differently than a mouse flow. The system of some embodiments detects an elephant flow by examining, among other items, the operations of a machine. The elephant flow represents a long-lived data flow or a continuous traffic flow that is associated with large data transfer. In some embodiments, the machine is a physical machine or a virtual machine (VM). In detecting, the system uses machine introspection to identify an initiation of a new data flow associated with the machine. The new data flow can be an outbound data flow or an inbound data flow. The system then tracks how much data the machine is sending or receiving through the connection, and determines, based on the amount of data being sent or received, if the data flow is an elephant flow.

(4) Different embodiments use different techniques to identify the initiation of a new flow of data that is associated with the machine. In some embodiments, the system identifies a new data flow by intercepting a network connection that is being opened on the machine. The connection can be an inbound network connection and/or an outbound network connection. In intercepting, the system of some embodiments performs a network introspection operation on the machine to intercept a socket call being made to open the new network connection.

(5) The system of some embodiments identifies the initiation of a new data flow by capturing a request to transfer (e.g., send or receive) a file. That is, rather than through a low-level socket call, the system of some embodiments detects an operating system (OS)/library or application programming interface (API) call to send or receive a file. The call may be associated with a particular network protocol for transferring files from one network host to another network host. Examples of different widely used data transfer protocols include file transfer protocol (FTP), Secure Shell (SSH) file transfer protocol, Bit Torrent, etc.

(6) In some embodiments, the system uses one of several different methods to determine the amount of data that is being transferred in a data flow. For some embodiments that detect elephant flows based on file transfer requests, the system makes this determination based on the size of the file that is being transferred. For instance, the system can identify the size of the file, and if the file is over a threshold size, the system can specify that the associated data flow is an elephant flow.

(7) Instead of identifying a file size, the system of some embodiments tracks the amount of data that has been transferred (e.g., sent or received). In some embodiments, the system tracks the data

size associated with every packet transferred in a data flow (e.g., through a network connection). For instance, the system may calculate the amount of data transferred by accumulating or adding the number of bytes transferred with each packet. If the number of bytes is over a threshold value, the system of some embodiments declares the data flow to be an elephant flow. In some embodiments, the system includes a machine that marks packet with a marking and a packet inspection agent uses the mark to track the amount of data sent and identify an elephant flow if the amount is over a threshold value.

(8) In conjunction with byte count or instead of it, the system of some embodiments factors in time. As an example, the system might detect an elephant flow solely based on how long the data flow has been associated with the machine. That is, if the duration of the data flow is over a set period of time, the system might determine that the data flow is an elephant flow. The duration can be calculated based on how long a network connection has been opened to handle the data flow. Also, instead of byte count, the process might calculate bit rate or bytes per second. The bit rate can be used to allow elephant flows with slow data transfer rate to progress as normal. This is because such elephant flows with slow transfer rate may not be contributing or at least significantly in the latency of other data flows, such as mice flows and non-detected elephant flows.

(9) The system of some embodiments identifies one or more pieces of information that provide context regarding the detected elephant flow. In some embodiments, the context information is used to identify the elephant flow. The context information may include user data, application data, and/or machine data. For instance, the system may identify the name of the source machine, the address (e.g., MAC address, IP address) associated with the source machine, the address associated with a destination machine, a port number (e.g., TCP port number, UDP port number), the application that initiated the call, user data (e.g., username of the person that is logged onto the machine), etc. In some embodiments, the source of the data flow is identified by source MAC address, source IP address, and source port. The destination may also be identified by same set of tuples or fields.

(10) Once an elephant flow is detected, the system of some embodiments treats the detected elephant flow differently than other flows (e.g., mouse flows, non-detected elephant flows). In some embodiments, the system reports the elephant flow and the associated context information (e.g., MAC address, IP address, etc.) to an agent that is interested in the elephant flow. For instance, the system may send a report to a forwarding element, such as a switch or router. The forwarding element may then use Quality of Service (QOS) configuration to place packets belonging to the elephant flow in a particular queue that is separate from one or more other queues with other packets. In this manner, one set of packets belonging to a mouse flow is not stuck in the same queue behind another set of packets belonging to an elephant flow.

(11) Alternatively, the system may send packets associated with an elephant flow along different paths (e.g., equal-cost multipath routing (ECMP) legs) to break the elephant flow into mice flows. As another example, the system may send elephant flow traffic along a separate physical network, such as an optical network that is more suitable for slow changing, bandwidth-intensive traffic. In some embodiments, the system reports the elephant flow to a network controller (e.g., a software-defined networking controller) that can configure one or more forwarding elements to handle the elephant flow.

(12) Additional techniques for detecting and handling elephant flows are described in U.S. patent application Ser. No. 14/231,647, entitled “Detecting and Handling Elephant Flows”, filed Mar. 31, 2014. Furthermore, several embodiments that detect an elephant flows based on the size of a packet are described in U.S. patent application Ser. No. 14/231,652, entitled “Detecting an Elephant Flow Based on the Size of a Packet”, filed Mar. 31, 2014. Some embodiments that report elephant flows to a network controller are described in U.S. patent application Ser. No. 14/231,654, entitled “Reporting Elephant Flows to a Network Controller, filed Mar. 31, 2014. These U.S Patent Applications are incorporated herein by reference.

(13) The preceding Summary is intended to serve as a brief introduction to some embodiments as described herein. It is not meant to be an introduction or overview of all subject matter disclosed in this document. The Detailed Description that follows and the Drawings that are referred to in the Detailed Description will further describe the embodiments described in the Summary as well as other embodiments. Accordingly, to understand all the embodiments described by this document, a full review of the Summary, Detailed Description and the Drawings is needed. Moreover, the claimed subject matters are not to be limited by the illustrative details in the Summary, Detailed Description and the Drawings, but rather are to be defined by the appended claims, because the claimed subject matters can be embodied in other specific forms without departing from the spirit of the subject matters.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

- (1) The novel features of the invention are set forth in the appended claims. However, for purposes of explanation, several embodiments of the invention are set forth in the following figures.
- (2) FIG. 1 illustrates an example of inspecting operations of a machine to detect an elephant flow.
- (3) FIG. 2 conceptually illustrates a process that some embodiments implement to detect an elephant flow.
- (4) FIG. 3 illustrates an example of inspecting operations of a machine to detect an elephant flow.
- (5) FIG. 4 illustrates a process that some embodiments implement to intercept a new connection for a data flow and determine whether the data flow is an elephant flow.
- (6) FIG. 5 illustrates an example of intercepting a high-level application call to detect an elephant flow.
- (7) FIG. 6 illustrates a process that some embodiments implement to intercept a high-level application call to transmit a file on a socket.
- (8) FIG. 7 a process that some embodiments implement to analyze context data relating to an application call to detect an elephant flow.
- (9) FIG. 8 illustrates an example of detecting an elephant flow using a combination of machine introspection and packet inspection.
- (10) FIG. 9 illustrates a process that some embodiments implement to intercept a new connection and to specify a unique marking for packets sent over the connection.
- (11) FIG. 10 illustrates a process that some embodiments implement to perform packet inspection in order to detect an elephant flow.
- (12) FIG. 11 conceptually illustrates an electronic system with which some embodiments of the invention are implemented.

DETAILED DESCRIPTION

- (13) In the following detailed description of the invention, numerous details, examples, and embodiments of the invention are set forth and described. However, it will be clear and apparent to one skilled in the art that the invention is not limited to the embodiments set forth and that the invention may be practiced without some of the specific details and examples discussed.
- (14) Embodiments described herein provide a system that detects whether a data flow is an elephant flow; and if so, the system treats it differently than a mouse flow. The system of some embodiments detects an elephant flow by examining, among other things, the operations of a machine. The elephant flow represents a long-lived data flow or a continuous traffic flow that is associated with large data transfer. In some embodiments, the machine is a physical machine or a virtual machine (VM). In detecting, the system identifies an initiation of a new data flow associated with the machine. The new data flow can be an outbound data flow or an inbound data flow. The system then determines, based on the amount of data being sent or received, if the data flow is an

elephant flow.

(15) For some embodiments of the invention, FIG. 1 illustrates an example of inspecting operations of a machine **100** to detect an elephant flow. Specifically, this figure illustrates in four stages **105-120** an example of (1) detecting an initiation or a new data flow, (2) determining the amount of data being sent or received in the data flow, and (3) specifying that the data flow is an elephant flow based on the amount of data. The figure includes the machine **100** that runs on the operating system (OS) **140**. The OS **140** includes a data flow interception agent **140**, an elephant detector **125**, and a network stack **130**. The figure also includes a forwarding element **135** that forwards packets for the machine **100**. The term “packet” is used here as well as throughout this application to refer to a collection of bits in a particular format sent across a network. It should be understood that the term “packet” may be used herein to refer to various formatted collections of bits that may be sent across a network, such as Ethernet frames, TCP segments, UDP datagrams, IP packets, etc.

(16) In the example of FIG. 1, the machine **100** can be a physical machine or a virtual machine (VM). Different from a physical or dedicated machine, the VM runs on a hypervisor of a host machine. The hypervisor can be a type 1 bare-metal hypervisor or a type 2 hosted hypervisor. For instance, the hypervisor can be a bare-metal hypervisor, which serves as a software abstraction layer that runs on top of the machine's hardware and runs below any OS.

(17) In some embodiments, the data flow interception agent **145** is a component of the OS **140** that is responsible for detecting an initiation of a new data flow associated with the machine **100**. The data flow can be an inbound data flow, which means that the data will be sent to the machine from another machine or network host. Alternatively, the data flow can be an outbound data flow, which means that the data will be sent from the machine to another machine. In detecting a new data flow, the data flow interception agent **145** may intercept each new network connection that is being initiated or opened on the machine. For instance, the data flow interception agent may perform a network introspection operation to intercept a socket call being made to open a new network connection to handle the data flow.

(18) Alternatively, the data flow interception agent **145** of some embodiments identifies each new data flow based on a request to send or receive a file. That is, rather than through a low-level socket call, the data flow interception agent detects an OS/library or application programming interface (API) call to transfer (e.g., send or receive) a file. The call may be associated with a particular protocol to transfer files from one network host to another network host. Examples of different widely used data transfer protocols include file transfer protocol (FTP), Secure Shell (SSH) file transfer protocol, Bit Torrent, etc.

(19) In some embodiments, the data flow interception agent **145** is a native component of the OS. That is, the data flow interception agent comes preinstalled with the OS. The data flow interception agent **145** may be a component that is installed on the OS. For instance, the data flow interception agent may be a thin agent or a part of a thin agent that is installed on the machine to perform network introspection. The thin agent may operate on the machine to intercept different types of events that are occurring on the machine. For instance, the thin agent may include a network introspection module to intercept each system call to open a new network connection, a file introspection module to intercept each system call to open a data file, etc.

(20) Different from the data flow interception agent **145**, the elephant detector **125** is responsible for determining whether the identified data flow is an elephant flow rather than a mouse flow. In determining, the elephant detector **125** of some embodiments identifies that the amount of data being sent or received is over a threshold value. The elephant detector can use one of several different methods to determine the amount of data that is being transferred with a data flow. For some embodiments that detect elephant flows based on file transfer requests, the elephant detector **125** makes this determination based on the size of the file that is being transferred in the data flow. For instance, the elephant detector **125** can identify the size of the file, and if the file is over the threshold value, the elephant detector can specify that the data flow is an elephant flow.

(21) Instead of identifying a file size, the elephant detector **125** of some embodiments tracks the amount of data that has been transferred (e.g., sent or received). In some embodiments, the elephant detector **125** tracks the data size associated with every packet transferred in a data flow (e.g., through a network connection). For instance, the elephant detector **125** may calculate the amount of data transferred by accumulating or adding the number of bytes transferred with each packet. If the number of bytes is over a threshold value, the elephant detector **125** of some embodiments declares the data flow to be an elephant flow.

(22) In conjunction with byte count or instead of it, the elephant detector **125** of some embodiments factors in time. As an example, the elephant detector **125** might detect an elephant flow solely based on how long the data flow has been associated with the machine. That is, if the duration of the data flow is over a set period of time, the elephant detector **125** might determine that the data flow is an elephant flow. The duration can be calculated based on how long a network connection has been opened to handle the data flow. Also, instead of byte count, the process might calculate bit rate or bytes per second. The bit rate can be used to allow elephant flows with slow data transfer rate to progress as normal. This is because such elephant flows with slow transfer rate may not be contributing or at least significantly in the latency of other data flows, such as mice flows and non-detected elephant flows.

(23) In the example of FIG. 1, the elephant detector **125** is shown as a component that runs on the OS **140** of the machine **100**. However, the elephant detector **125** may be a part of a hypervisor that executes on a host machine, a component or module that executes on a separate machine (e.g., virtual or physical machine), or a dedicated appliance.

(24) The network stack **130** conceptually represents the OS's implementation of a number of different protocols to send and receive data through a network. The network stack **130** may be a part of the OS's kernel. The network stack, such as the TCP/IP network stack, is used to process data through several different layers. For instance, when outputting data from the machine, the data may be sent to a socket buffer and processed at the TCP layer to create TCP segments or packets. Each segment is then processed by a lower layer, such as the IP layer to add an IP header. The output of the network stack is a set of packets associated with the outbound data flow. Each packet may be processed (e.g., segmented) by the machine's network interface card, and sent to the forwarding element **135**. On the other hand, when receiving data at the machine, each packet may be processed by one or more of the layers in reverse order to strip one or more headers, and place the user data or payload in an input socket buffer.

(25) In some embodiments, the forwarding element **135** is a hardware-forwarding element. The hardware-forwarding element can have application-specific integrated circuits (ASICs) that are specifically designed to support in-hardware forwarding. Alternatively, the physical forwarding element **135** can be a software-forwarding element, such as Open vSwitch (OVS). In some embodiments, the forwarding element (e.g., software or hardware forwarding element) is a physical forwarding element that operates in conjunction with one or more other physical forwarding elements to collectively implement different logical forwarding elements (e.g., logical switches, logical routers, etc.) for different logical networks of different tenants, users, departments, etc. that use the same shared computing and networking resources. Accordingly, the term "physical forwarding element" is used herein to differentiate it from a logical forwarding element.

(26) In some embodiments, the forwarding element **135** is an edge forward element (EFE). In some embodiments, the edge forwarding element represents a last forwarding element before one or more end machines (e.g., the machine **100**). Alternatively, the forwarding element **135** can be a non-edge forwarding (NEFE). Irrespective of whether the forwarding element **135** is positioned at the edge of the network or not, the forwarding element is configured to treat packets associated with a detected elephant flow differently than other packets associated with other data flows (e.g., mouse flows, non-detected elephant flows).

(27) Having described several components of FIG. 1, example operations of these components will

now be described by reference to the four stages **105-120** that are illustrated in the figure. The first stage **105** shows that the machine **100** is communicatively coupled to the forwarding element **135**. The data flow interception agent **145** operates on the OS **140** of the machine **100** to detect an initiation of a new data flow. In the first stage **105**, the data flow interception agent **145** has detected a new data flow being initiated. The data flow interception agent **145** might have made the detection by intercepting a call to open or use a particular network connection to handle the new data flow. In some embodiments, the data flow interception agent **145** detects a new data flow by intercepting an application request to send or receive a file.

(28) The second stage **110** shows the data flow interception agent **145** sending context data associated with the data flow to the elephant detector **125**. As mentioned above, the context information can include user data, application data, and/or machine data. For instance, the system may identify the name of the source machine, the address (e.g., MAC address, IP address) associated with the source machine, the address associated with a destination machine, a port number associated the source and/or destination, the application that initiated the call, user data (e.g., username of the person that is logged onto the machine, etc. In some embodiments, the source is identified as a combination of source IP address, and port number; and the destination is identified by a combination of destination IP address and port number.

(29) The second stage **110** also shows that the new data flow has been started. This is conceptually shown with several packets being forwarded by the forwarding element **135**.

(30) The third stage **115** shows the elephant detector **125** determining that the data flow is an elephant flow. As mentioned above, the elephant flow detector **125** can make this determination based on the size of the file that is being transferred in the data flow. For instance, the elephant detector can identify the size of the file; and if the file is over the threshold value, the elephant detector can specify that the data flow is an elephant flow. Instead of identifying a file size, the elephant detector may track the amount of data that has been transferred. For instance, the elephant detector may calculate the amount of data transferred by accumulating or adding the number of bytes transferred with each packet.

(31) The fourth stage **120** shows an example operation of the elephant detector **125** upon detecting the elephant flow. Here, the elephant flow detector **125** reports the elephant flow to the forwarding element **135**. The forwarding element **135** may then use Quality of Service (QOS) configuration to place packets belonging to the elephant flow in a particular queue that is separate from one or more other queues with other packets). In this manner, one set of packets belonging to a mouse is not stuck in the same queue behind another set of packets belonging to an elephant.

(32) Alternatively, the forwarding element **135** may send packets associated with an elephant flow along different paths (e.g., equal-cost multipath routing (ECMP) legs) to break the elephant flow into mice flows. As another example, the forwarding element **135** may send elephant flow traffic along a separate physical network, such as an optical network that is more suitable for slow changing, bandwidth-intensive traffic. In some embodiments, the elephant detector reports the elephant flow to a network controller (e.g., a software-defined networking controller) that can configure one or more forwarding elements (e.g., the forwarding element **100**) to handle the elephant flow.

(33) Having described an example of detecting an elephant flow, a process will now be described by reference to FIG. 2. FIG. 2 conceptually illustrates a process **200** that some embodiments implement in order to detect elephant flows. In some embodiments, the process **200** is performed by one or more components shown in FIG. 1, such as the data flow interception agent **145** and the elephant detector **125**.

(34) As shown in FIG. 2, the process **200** begins when it identifies (at **205**) a new flow of data that is associated with a machine. The data flow can be an inbound or outbound data flow. The process **200** then determines (at **210**) whether the amount of data being sent or received is greater than a threshold value. If the amount is not over, the process **200** ends. However, if the amount is over, the

process **200** specifies (at **215**) that the data flow is an elephant flow. The process **200** then reports (at **220**) the elephant flow to an agent that is interested in the report. The agent that receives a message regarding the elephant flow may be operating on the same operating system (OS) as the elephant detector, the same machine (e.g., as a part of a virtual switch, as a part of a hypervisor, as part of a service virtual machine), or another machine or device (e.g., as part of a network controller which controls one or more software or hardware forwarding elements, as a part of hardware switch, as part of a dedicated appliance, etc.). In some embodiments, the agent on the same machine facilitates in marking packets associated with the elephant flow with a particular mark. As an example, the agent may mark each packet associated with an elephant flow using a Differentiated Services Code Point (DSCP) field that provide different levels of service to be assigned to network traffics, such as IP packets. The process **200** then ends.

(35) In some embodiments, the process **200** performs other calculation to determine whether a data flow is an elephant flow. In conjunction with byte count or instead of it, the process **200** of some embodiments factors in time. As an example, the process **200** might detect an elephant flow solely based on how long the data flow has been associated with the machine. That is, if the duration of the data flow is over a set period of time, the process **200** might determine that the data flow is an elephant flow. The duration can be calculated based on how long a network connection has been opened to handle the data flow. Also, instead of byte count, the process **200** might calculate bit rate or bytes per second. The bit rate can be used to allow elephant flows with slow data transfer rate to progress as normal. This is because such elephant flows with slow transfer rate may not be contributing or at least significantly in the latency of other data flows.

(36) Some embodiments perform variations on the process **200**. The specific operations of the process **200** may not be performed in the exact order shown and described. The specific operations may not be performed in one continuous series of operations, and different specific operations may be performed in different embodiments.

(37) Several more examples of detection and handling elephant flows will be described in detail below. Section I describes several additional examples of detecting elephant flows based on machine introspection. In particular, Section I.A describes an example of detecting a new data flow by intercepting a network connection that is being opened on the machine. Section I.B then describes an example of detecting a new data flow by capturing a request to transfer a file. This is followed by Section I.C, which describes an example of detecting an elephant flow using a combination of machine introspection and packet inspection. Section II then describes an example electronic system with which some embodiments of the invention are implemented.

(38) I. Examples of Detecting Elephant Flows based on Machine Introspection

(39) In some embodiments, the system detects an elephant flow by monitoring, among other things, the operations of a machine. In detecting, the system uses machine introspection to intercept a new network connection that is being initiated on the machine. The system then identifies context information associated with the connection. The context information can include one or more of the following: the name of the source machine, the address (e.g., MAC address, IP address) associated with the source machine, the address associated with a destination machine, a port number for the source and/or destination, the application that initiated the call, user data (e.g., username of the person that is logged onto the machine), etc. The system then uses machine introspection to track how much data the machine is sending or receiving through the connection. The system then determines, based on the amount of data being sent or received, whether the data flow associated with the connection is an elephant flow. If the system detects an elephant flow, it reports the elephant flow and the associated context information to an agent (e.g., a forwarding element, a network controller) that is interested in the elephant flow.

(40) The system of some embodiments can also, or alternatively, detect an elephant flow based on high-level application calls, rather than low-level system calls. As an example, the system might detect an OS/library or application programming interface (API) call to send or receive a file. The

system then determines whether an elephant flow is associated with the call by identifying the size of the file that is being transferred. If the size is greater than a threshold value, the system then reports the elephant flow to the agent. In some embodiments, the system detects an elephant flow using a combination of machine introspection and packet inspection. Several such examples will now be described below by reference to FIGS. 3-10.

(41) A. Introspecting Control Path and Data Path

(42) FIG. 3 illustrates an example of inspecting operations of a machine to detect an elephant flow. Specifically, this figure shows in four stages 301-304 how an introspection agent 345 on a machine 305 detects a new network connection and reports the new connection to an elephant detector 340. The elephant detector 340 then determines the amount of data transferred with the network connection. If the amount is over a threshold value, the elephant flow detector 340 sends a message regarding the elephant flow to any party or agent that is interested in such a report.

(43) In the example of FIG. 3, the machine 305 is a guest virtual machine (VM) that executes on a host machine 300. However, it should be understood that the example shown in the figure is equally applicable to a physical machine. As shown, the introspection agent 345 is a thin agent that is installed on the operating system (OS) of the guest VM 305. The elephant detector 340 is a component that operates on the host machine 300. For instance, the elephant detector 340 may be a component that executes on a separate VM than the guest VM 305. The figure also shows a number of applications 310-320. These applications could be any different type of applications that send data over a network.

(44) In some embodiments, the thin agent 345 operates on the VM 305 and intercepts different types of events that are occurring on the VM. From a control path point of view, the thin agent 345 may intercept a new network connection being made and/or a file being opened. For instance, when a user sends a file through an application (310, 315, or 320), the thin agent 345 may detect that the application has made a socket call to open a new connection.

(45) To trap control data events, the thin agent 345 of some embodiments includes a set of one or more control path interceptors 325. One example of a control path interceptor 325 is a network introspection module that intercepts socket calls being made on the VM 305 to open a new network connection. For each intercepted call, the network introspection module may identify various pieces of information (e.g., contextual information) associated with the call. The network introspection module may identify the name of the VM, the address (e.g., MAC address, IP address) associated with the VM, the address associated with the destination machine, a port number for the source and/or destination, the application that initiated the call, user data (e.g., username of the person that is logged onto the VM), etc. The thin agent 345 can include different types of control path interceptors to perform machine introspection, such as a file introspection module that detects calls to open different files.

(46) To trap information on the data path, the thin agent 345 of some embodiments includes a set of one or more data path interceptors 330. In some embodiments, a data path interceptor 330 identifies the size of data associated with each packet transferred over the network connection. In some embodiments, the data size is the size of the payload or user data of the packet. Alternatively, the data size may be the actual size of the packet including one or more protocol headers and trailers. In some embodiments, the data path interceptor 330 reports the data size associated with each packet to the elephant detector 340. To optimize processing, the data path interceptor may report the data size once it reaches a certain limit, in some embodiments.

(47) The elephant detector 340 of some embodiments tracks the amount of data that has been transferred (e.g., sent or received) with the network connection. In determining the amount of data being sent or received, the elephant detector may receive the data size of each packet with a unique identifier for the data flow or the network connection. If the data amount reaches a threshold value, the elephant detector then correlates the unique identifier with the context information. Thereafter, the elephant detector may report the elephant flow and the associated context information.

(48) In some embodiments, the thin agent **345** or another component on the guest VM **305** performs the data size aggregation and elephant flow detection. For instance, if the thin agent is aware of the threshold value, the thin agent can add the data size associated with each packet to determine whether the amount of data transferred is greater than the threshold value.

(49) As shown in FIG. 3, a guest introspection host module **335** (hereinafter referred to as a multiplexer (MUX)) operates on the host **300**. The MUX **335** receives intercepted data from the thin agent **345**. The MUX **335** then forwards the intercepted data to one or more components, appliances, and/or devices that are interested in that data. For example, a component operating on the host **300** may be registered with the MUX **335** to receive an asynchronous notification each time a certain type of control path event is intercepted by the thin agent **345** on the VM **305**.

(50) In some embodiments, the elephant detector **340** detects elephant flows using information provided by the thin agent **345** via the MUX **335**. In the example of FIG. 3, the elephant detector is shown as a component that execute on the host **300**. For instance, the elephant detector **340** may be a component that executes on a separate VM than the guest VM **305**. Alternatively, the elephant detector **340** may be a part of a hypervisor that executes on the host, a component that executes on a separate physical machine, or a dedicated appliance.

(51) Having described example components of FIG. 3, example operations of these components will now be described by reference to the four stages **301-304** that are illustrated in the figure. The first stage **301** shows the VM **305** executing on the host **300**. The thin agent **345** has been installed on the OS of the VM **305**. The control path interceptor **325** is registered to intercept calls to create new network connections. The thin agent **345** may have previously updated information regarding the user that is logged onto the OS. The user information is updated because that the thin agent **345** of some embodiments reports the information with each intercepted event.

(52) The second stage **302** shows the thin agent **345** intercepting a request made by the application **310** to open a new connection. In particular, the control path interceptor **325** intercepts a socket call and identifies information regarding the socket call. The control path interceptor **325** may identify the application **310** that initiated the call, user data, the type of connection (e.g., inbound or outbound), the address (e.g., MAC address, IP address) associated with the VM, the address associated with the destination machine, a port number, etc.

(53) The second stage **302** also shows the thin agent **345** facilitating in detecting the amount of data being sent or received through the network connection. As mentioned above, the data path interceptor **330** reports the file or data size associated with each packet to the elephant detector **340**. To optimize processing, the data path interceptor may report the data size once it reaches a certain limit, in some embodiments.

(54) The third stage **303** illustrates the thin agent **345** sending data to the MUX **335** regarding the new connection. The thin agent **345** of some embodiments sends the data upon trapping the data. That is, the thin agent **345** does not wait for a specified period of time but sends the data immediately to the MUX **335**. The thin agent **345** might format the data in a particular format prior to sending it to the MUX **335**. For instance, the thin agent might encode the data in JavaScript Object Notation (JSON) or Extensible Markup Language (XML). In some embodiments, the thin agent **345** maintains a local copy of each event. For a given session, the thin agent **345** may maintain a log of all intercepted events. The local copy provides a backup in case there is a communication error between the thin agent **345** and the MUX **335**.

(55) The fourth stage **304** illustrates the MUX **335** sending the data to the elephant detector **340**. Here, the MUX **335** provides the data asynchronously or synchronously to the elephant detector **340**. The MUX **335** may also store the data in storage (not shown) prior to sending the data to the elephant detector **340**. The MUX **335** may receive the data in one format and send the data in the same format or different format to the elephant detector **340**, in some embodiments.

(56) In the fourth stage **304**, the elephant detector **340** receives the data from the MUX **335**. The elephant detector **340** then uses the data to track the amount of data flowing through the network

connection. If the amount exceeds a threshold value, the elephant detector **340** may specify that the data flow associated with the connection is an elephant flow. The elephant detector **340** may then report the elephant flow to agent or party that is interested in the report. For instance, in some embodiments, the elephant detector reports the elephant flow to a network controller, which in turn configures one or more forwarding elements to handle the elephant flow. Alternatively, the report may be sent directly to a forwarding element.

(57) In the example described above, the thin agent **345** intercepts the initiation of a new connection, and gathers data relating to the connection (e.g., data size of each packet, context data). The thin agent **345** then provides the data to the MUX **335**, which in turn provides the data to the elephant flow detector **340**.

(58) FIG. **4** illustrates a process **400** that some embodiments implement to intercept a new connection for a data flow and determine whether the data flow is an elephant flow. In some embodiments, the process **400** is performed by one or more components shown in FIG. **3**, such as the thin agent **345** and the elephant detector **340**.

(59) As shown in FIG. **4**, the process **400** begins when it registers (at **405**) to intercept the initiation of any new network connections (e.g., outbound network connections, inbound network connections). The process **400** then begins detecting whether a new connection has been initiated. For instance, the process **400** of some embodiments detects (at **410**) whether a socket call has been made to open a new connection. If such a call has been made, the process **400** identifies (at **415**) the application, the user, VM context information associate with the connection. The process **400** then tracks (at **420**) the amount of data being transferred (e.g., sent or received) over the network connection.

(60) At **425**, the process **400** determines whether the amount of data being sent or received is greater than a threshold value. If the amount is over, the process **400** specifies (at **430**) that the data flow is an elephant flow. The process **400** then reports (at **435**) the elephant flow (e.g., to a forwarding element and/or a network controller). The agent that receives a message regarding the elephant flow may be operating on the same operating system (OS) as the elephant detector **340**, the same machine (e.g., as a part of a virtual switch, as a part of a hypervisor, as part of a service virtual machine, etc.) or another machine or device (e.g., as part of a network controller which controls one or more software or hardware forwarding elements, as a part of hardware switch, as part of a dedicated appliance, etc.). In some embodiments, the agent on the same machine facilitates in marking packets associated with the elephant flow with a particular mark. As an example, the agent may mark each packet associated with an elephant flow using a Differentiated Services Code Point (DSCP) field that provide different levels of service to be assigned to network traffics, such as IP packets. If the amount of data transferred is not greater than the threshold value, the process **400** assumes that the data flow associated with the connection is a mouse flow and does not report it. The process **400** then ends.

(61) Some embodiments perform variations on the process **400**. The specific operations of the process **400** may not be performed in the exact order shown and described. The specific operations may not be performed in one continuous series of operations, and different specific operations may be performed in different embodiments. For instance, the tracking operation **420** may be performed by different components, such as the thin agent or the elephant detector. As mentioned above, if the thin agent is aware of the threshold value, the thin agent may aggregate the data size and perform the elephant flow detection. Also, the thin agent might send data to the elephant agent through one or more intermediary agents (e.g., the MUX).

(62) B. High-Level Application Calls

(63) In the example described above, the thin agent intercepts a socket call to open a new network connection. The system of some embodiments can detect an elephant flow based on an application call for a new file transfer. As an example, the system might detect an OS/library or application programming interface (API) call to send or receive a file. The call may be associated with a

particular network protocol for transferring files from one network host to another network host. Examples of different widely used data transfer protocols include file transfer protocol (FTP), Secure Shell (SSH) file transfer protocol, Bit Torrent, etc. The system then determines whether an elephant flow is associated with call by identifying the size of the file that is being transferred with the call. If the size is greater than a threshold value, the system then reports the elephant flow to an agent that is interested in the report.

(64) FIG. 5 illustrates an example of intercepting a high-level application call to detect an elephant flow. Four operational stages **505-520** of the host **500** are shown in this figure. This figure is similar to FIG. 3, except that the control and data path interceptors **325** and **330** have been replaced by a set of API call interceptors **525**.

(65) The first stage **505** shows the VM **530** executing on the host **500**. The thin agent **345** has been installed on the OS of the VM **530**. The API call interceptor **525** is registered to intercept high-level application calls to send or receive data over a network. The thin agent **345** may have previously updated information regarding the user that is logged onto the OS. The user information is updated because that the thin agent **345** of some embodiments reports the information with each intercepted event.

(66) The second stage **510** shows the thin agent **345** intercepting an API call made by the application **310**. In particular, the API call interceptor **525** intercepts an API call to transmit a file on a socket. The API call interceptor **525** may identify the application **310** that initiated the call, user data, the type of connection (e.g., inbound or outbound), the address (e.g., MAC address, IP address) associated with the VM, the address associated with the destination machine, a port number, etc.

(67) The second stage **510** also shows the thin agent **345** identifying the size of the data that is to be sent or received over the new connection. The third stage **515** illustrates the thin agent **345** sending data to the MUX **335** regarding the API call. The fourth stage **520** illustrates the MUX **335** sending the data to the elephant detector **340**.

(68) In the fourth stage **520**, the elephant detector **340** receives the data from the MUX **335**. Having received the data, the elephant detector **340** then analyzes the data to detect an elephant flow. In some embodiments, the elephant detector **340** receives the size of the data that is being transferred over the network connection with the API call. The elephant detector **340** may compare the size of the data to a threshold value. If the size exceeds the threshold value, the elephant detector **340** may specify that the data flow associated with the API call is an elephant flow. The elephant detector **340** may then report the elephant flow to agent or party that is interested in the report. For instance, in some embodiments, the elephant detector reports the elephant flow to a network controller, which in turn configures one or more forwarding elements to handle the elephant flow. Alternatively, the report may be sent directly to a forwarding element.

(69) In the example described above, the thin agent **345** intercepts an API call and the size of file that is to be downloaded to or uploaded from the machine. The thin agent **345** then provides the data to the MUX **335**, which in turn provides the data to the elephant flow detector **340**. FIG. 6 illustrates a process **600** that some embodiments implement to intercept a high-level application call to transmit a file on a socket. This is followed by FIG. 7, which illustrates a process **700** that some embodiments implement to analyze the data to detect an elephant flow. In some embodiments, the process **600** of FIG. 6 is performed by the thin agent **345**, and the process **700** of FIG. 7 is performed by the elephant detector **340**.

(70) As shown in FIG. 6, the process **600** begins when it registers (at **605**) to intercept API calls to send or receive a file on a socket. The process **600** then begins detecting (at **610**) whether a new file transfer has been initiated (e.g., a new file is being sent or received). If a detection has been made, the process **600** identifies (at **615**) the application, the user, VM context information associated with the connection. The process **600** then identifies (at **620**) the size of the file that is being transferred with the call. The process **600** then reports (at **625**) the context information and

the size of the data (e.g., to the MUX). The process **600** then ends.

(71) As shown in FIG. 7, the process **700** begins when it receives (at **705**) the context info and the size of file. Here, the dashed arrow between the two processes **600** and **700** conceptually illustrate that the elephant agent might have received the data through an intermediary agent (e.g., the MUX). At **710**, the process **700** determines whether the size of the file is greater than a threshold value. If so, the process **700** specifies (at **715**) that the data flow associated with the API call is as an elephant flow.

(72) At **720**, the process **700** reports the elephant flow (e.g., to a forwarding element and/or a network controller). If the size of the file is not greater than a threshold value, the process **700** assumes that the data flow associated with the connection is a mouse flow and does not report it. The process **700** then ends. Some embodiments perform variations on the processes **600** and **700**. The specific operations of the processes **600** and **700** may not be performed in the exact order shown and described. The specific operations may not be performed in one continuous series of operations, and different specific operations may be performed in different embodiments.

(73) C. Combination of Machine Introspection and Packet Inspection

(74) In some embodiments, the system detects an elephant flow using a combination of machine introspection and packet inspection. In detecting, the system may intercept a socket call that is being initiated by an application, or may intercept a high-level API call. In determining the amount of data being sent, the system of some embodiments specifies a unique identifier for the network connection. The system then marks outbound data (e.g., packets) with a marking (e.g., a unique marking) and keeps track of the amount of data using the marking. If the amount of data sent reaches a threshold value, the system then correlates the unique identifier of the outbound traffic with the context information. Thereafter, the system uses the context information to relay a message regarding the detected elephant flow (e.g., to an agent that is interested in the report).

(75) FIG. 8 illustrates an example of detecting an elephant flow using a combination of machine introspection and packet inspection. In this example, the machine introspection is performed by the thin agent **345** executing on the VM **835**. The packet inspection is performed by a hypervisor **825**, namely the hypervisor's packet inspection module **830**.

(76) In this example, the hypervisor **825** is a bare metal hypervisor that runs on top of the hardware of the host machine **800** and runs below any operating system (OS). The hypervisor **825** handles various management tasks, such as memory management, processor scheduling, or any other operations for controlling the execution of the VM **835**. Moreover, the hypervisor **825** can communicate with the VM **835** to achieve various operations (e.g., setting priorities). In some embodiments, the hypervisor **825** is one type of hypervisor (Xen, ESX, or KVM hypervisor) while, in other embodiments, the hypervisor **825** may be any other type of hypervisor for providing hardware virtualization of the hardware on the host **800**.

(77) To perform packet inspection, the hypervisor **825** includes the packet inspection module **830**. For the purposes of elephant flow detection, the packet inspection module **830** is used to determine the amount of data sent over a particular connection. For instance, the packet inspection module of some embodiments keeps track of the number of packet sent (e.g., byte count and/or packet count) with a particular identifier (e.g., unique connection ID). In some embodiments, the packet inspection module uses packet header data to track the amount of data sent or received. For instance, the packet inspection module might identify the size of the payload of a packet from a header field. If the amount of data sent reaches a threshold value, the system then correlates the particular identifier with the context information associated with the connection. Thereafter, the hypervisor **825** uses the context information to report the elephant flow.

(78) Having described example components of FIG. 8, example operations of these components will now be described by reference to the four stages **805-820** that are illustrated in the figure. The first stage **805** shows the VM **835** executing on the host **800**. The thin agent **345** has been installed on the OS of the VM **835**. The control path interceptor **325** is registered to intercept calls to create

new network connections. The thin agent **345** may have previously updated information regarding the user that is logged onto the OS. The user information is updated because that the thin agent **345** of some embodiments reports the information with each intercepted event.

(79) The second stage **810** shows the thin agent **345** intercepting a request made by the application **310** to open a new connection. In particular, the control path interceptor **325** intercepts a socket call and identifies information regarding the socket call. The control path interceptor **325** may identify the application **310** that initiated the call, user data, the type of connection (e.g., inbound or outbound), the address (e.g., MAC address, IP address) associated with the VM, the address associated with the destination machine, a port number, etc.

(80) The third stage **815** illustrates the thin agent **345** sending data to the MUX **335** regarding the new connection. Prior to sending the data, the thin agent of some embodiments specifies an identifier (e.g., a unique identifier) for the connection. This unique identifier allows the packet inspection module **830** to keep track of the number of bytes sent and/or the number of packets sent thorough the connection.

(81) The fourth stage **820** illustrates the MUX **335** sending the data to the hypervisor **825**. Here, the MUX **335** provides the data asynchronously or synchronously to the hypervisor **825**. In the fourth stage **820**, the hypervisor **825** receives the data from the MUX **335**. Having received the data, the packet inspection module then keeps track of the data sent through the connection. For example, the packet inspection module may store statistics relating to the packets with a particular marking. If the amount of data sent reaches a threshold value, the hypervisor **825** then correlates the particular marking with the context information associated with the connection. The hypervisor **825** then uses the context information to report the elephant flow. Here, the control path interceptor provides the context to the packet inspection module via the MUX. Also, each packet is stamped with an ID. This ID is then used to correlate the data being transferred with a connection (e.g., initiated by an application or user) and identify the elephant flow. In some embodiments, the hypervisor **825** reports the elephant flow to a network controller, which in turn configures one or more forwarding elements to handle the elephant flow. Alternatively, the report may be sent directly to a forwarding element.

(82) In the example described above, the thin agent **345** intercepts the initiation of a new connection and the hypervisor identifies the amount of data that has been sent through the connection. FIG. **9** illustrates a process **900** that some embodiments implement to intercept a new connection and trap information regarding the connection. This is followed by FIG. **10**, which illustrates a process **1000** that some embodiments implement to analyze the data to detect an elephant flow. In some embodiments, the process **900** of FIG. **9** is performed by the thin agent **345**, and the process **1000** of FIG. **10** is performed by the hypervisor **825**.

(83) As shown in FIG. **9**, the process **900** begins when it registers (at **905**) to intercept the initiation of any new network connections (e.g., outbound network connections). The process **900** then begins detecting whether a new connection has been initiated. For instance, the process **900** of some embodiments detects (at **910**) whether a socket call has been made to open a new connection. If a detection has been made, the process **900** identifies (at **915**) the application, the user, and/or VM context information associate with the connection. The process then then defines or specifies (at **920**) a unique marking to mark packets that are sent through the connection. The process **900** then reports (at **925**) the context information and the marking. For instance, in the example described above, the control path interceptor provides the context to the packet inspection module via the MUX. Also, each packet is stamped with a marking. The process **900** then ends. In some embodiments, the process **900** returns to operation **920** when it detects another new connection.

(84) As shown in FIG. **10**, the process **1000** begins when it receives (at **1005**) the context info and the marking. Here, the dashed arrow between the two processes **900** and **1000** conceptually illustrate that the hypervisor might have received the data through an intermediary agent (e.g., the MUX).

(85) At **1010**, the process **1000** starts inspecting packets sent through the connection using the marking. The marking relates the packets to the connection. In particular, the process **1000** identifies (at **1010**) a packet with the marking. Based on the identified packet, the process **1000** determines (at **1015**) whether the amount of data sent is greater than a threshold value. If so, the process **1000** specifies (at **1025**) that the data flow associated with the connection is as an elephant flow. That is, the marking is used to correlate the data being transferred with a connection (e.g., initiated by an application or user) and identify the elephant flow. The process **1000** then reports (at **1030**) the elephant flow to an agent that is interested in the report. If the size of the data is not greater than a threshold value, the process **1000** proceeds to **1020**, which is described below.

(86) The process **1000** determines (at **1020**) whether there is another packet being sent through the connection. If there is another packet, the process **1000** returns to **1010**, which is described above. Otherwise, the process **1000** ends. Some embodiments perform variations on the processes **900** and **1000**. The specific operations of the processes **900** and **1000** may not be performed in the exact order shown and described. The specific operations may not be performed in one continuous series of operations, and different specific operations may be performed in different embodiments.

(87) For instance, in the example described above, the process **1000** inspects packet sent through a connection. In some embodiments, the process might perform similar operations for incoming packets. However, one or more of the detection rules are applied only based on the destination identity. This is because the process may only have the destination identity information and do not have the source identify information. As an example, the process might identify the destination IP address and destination port, and track the amount of data sent to the destination machine.

(88) II. Electronic System

(89) Many of the above-described features and applications are implemented as software processes that are specified as a set of instructions recorded on a computer readable storage medium (also referred to as computer readable medium). When these instructions are executed by one or more computational or processing unit(s) (e.g., one or more processors, cores of processors, or other processing units), they cause the processing unit(s) to perform the actions indicated in the instructions. Examples of computer readable media include, but are not limited to, CD-ROMs, flash drives, random access memory (RAM) chips, hard drives, erasable programmable read-only memories (EPROMs), electrically erasable programmable read-only memories (EEPROMs), etc. The computer readable media does not include carrier waves and electronic signals passing wirelessly or over wired connections.

(90) In this specification, the term “software” is meant to include firmware residing in read-only memory or applications stored in magnetic storage, which can be read into memory for processing by a processor. Also, in some embodiments, multiple software inventions can be implemented as sub-parts of a larger program while remaining distinct software inventions. In some embodiments, multiple software inventions can also be implemented as separate programs. Finally, any combination of separate programs that together implement a software invention described here is within the scope of the invention. In some embodiments, the software programs, when installed to operate on one or more electronic systems, define one or more specific machine implementations that execute and perform the operations of the software programs.

(91) FIG. **11** conceptually illustrates an electronic system **1100** with which some embodiments of the invention are implemented. The electronic system **1100** may be a computer (e.g., a desktop computer, personal computer, tablet computer, etc.), server, dedicated switch, phone, PDA, or any other sort of electronic or computing device. Such an electronic system includes various types of computer readable media and interfaces for various other types of computer readable media. Electronic system **1100** includes a bus **1105**, processing unit(s) **1110**, a system memory **1125**, a read-only memory **1130**, a permanent storage device **1135**, input devices **1140**, and output devices **1145**.

(92) The bus **1105** collectively represents all system, peripheral, and chipset buses that

communicatively connect the numerous internal devices of the electronic system **1100**. For instance, the bus **1105** communicatively connects the processing unit(s) **1110** with the read-only memory **1130**, the system memory **1125**, and the permanent storage device **1135**.

(93) From these various memory units, the processing unit(s) **1110** retrieves instructions to execute and data to process in order to execute the processes of the invention. The processing unit(s) may be a single processor or a multi-core processor in different embodiments.

(94) The read-only-memory (ROM) **1130** stores static data and instructions that are needed by the processing unit(s) **1110** and other modules of the electronic system. The permanent storage device **1135**, on the other hand, is a read-and-write memory device. This device is a non-volatile memory unit that stores instructions and data even when the electronic system **1100** is off. Some embodiments of the invention use a mass-storage device (such as a magnetic or optical disk and its corresponding disk drive) as the permanent storage device **1135**.

(95) Other embodiments use a removable storage device (such as a floppy disk, flash memory device, etc., and its corresponding drive) as the permanent storage device. Like the permanent storage device **1135**, the system memory **1125** is a read-and-write memory device. However, unlike storage device **1135**, the system memory **1125** is a volatile read-and-write memory, such a random access memory. The system memory **1125** stores some of the instructions and data that the processor needs at runtime. In some embodiments, the invention's processes are stored in the system memory **1125**, the permanent storage device **1135**, and/or the read-only memory **1130**. From these various memory units, the processing unit(s) **1110** retrieves instructions to execute and data to process in order to execute the processes of some embodiments.

(96) The bus **1105** also connects to the input and output devices **1140** and **1145**. The input devices **1140** enable the user to communicate information and select commands to the electronic system. The input devices **1140** include alphanumeric keyboards and pointing devices (also called "cursor control devices"), cameras (e.g., webcams), microphones or similar devices for receiving voice commands, etc. The output devices **1145** display images generated by the electronic system or otherwise output data. The output devices **1145** include printers and display devices, such as cathode ray tubes (CRT) or liquid crystal displays (LCD), as well as speakers or similar audio output devices. Some embodiments include devices such as a touchscreen that function as both input and output devices.

(97) Finally, as shown in FIG. **11**, bus **1105** also couples electronic system **1100** to a network **1165** through a network adapter (not shown). In this manner, the computer can be a part of a network of computers (such as a local area network ("LAN"), a wide area network ("WAN"), or an Intranet, or a network of networks, such as the Internet. Any or all components of electronic system **1100** may be used in conjunction with the invention.

(98) Some embodiments include electronic components, such as microprocessors, storage and memory that store computer program instructions in a machine-readable or computer-readable medium (alternatively referred to as computer-readable storage media, machine-readable media, or machine-readable storage media). Some examples of such computer-readable media include RAM, ROM, read-only compact discs (CD-ROM), recordable compact discs (CD-R), rewritable compact discs (CD-RW), read-only digital versatile discs (e.g., DVD-ROM, dual-layer DVD-ROM), a variety of recordable/rewritable DVDs (e.g., DVD-RAM, DVD-RW, DVD+RW, etc.), flash memory (e.g., SD cards, mini-SD cards, micro-SD cards, etc.), magnetic and/or solid state hard drives, read-only and recordable Blu-Ray® discs, ultra density optical discs, any other optical or magnetic media, and floppy disks. The computer-readable media may store a computer program that is executable by at least one processing unit and includes sets of instructions for performing various operations. Examples of computer programs or computer code include machine code, such as is produced by a compiler, and files including higher-level code that are executed by a computer, an electronic component, or a microprocessor using an interpreter.

(99) While the above discussion primarily refers to microprocessor or multi-core processors that

execute software, some embodiments are performed by one or more integrated circuits, such as application specific integrated circuits (ASICs) or field programmable gate arrays (FPGAs). In some embodiments, such integrated circuits execute instructions that are stored on the circuit itself. In addition, some embodiments execute software stored in programmable logic devices (PLDs), ROM, or RAM devices.

(100) As used in this specification and any claims of this application, the terms “computer”, “server”, “processor”, and “memory” all refer to electronic or other technological devices. These terms exclude people or groups of people. For the purposes of the specification, the terms display or displaying means displaying on an electronic device. As used in this specification and any claims of this application, the terms “computer readable medium,” “computer readable media,” and “machine readable medium” are entirely restricted to tangible, physical objects that store information in a form that is readable by a computer. These terms exclude any wireless signals, wired download signals, and any other ephemeral signals.

(101) While the invention has been described with reference to numerous specific details, it should be understood that the invention can be embodied in other specific forms without departing from the spirit of the invention. In addition, a number of the figures (including FIGS. 2, 4, 6, 7, 9, and 10) conceptually illustrate processes. The specific operations of these processes may not be performed in the exact order shown and described. The specific operations may not be performed in one continuous series of operations, and different specific operations may be performed in different embodiments. Furthermore, the process could be implemented using several sub-processes, or as part of a larger macro process. Thus, it should be understood that the invention is not to be limited by the foregoing illustrative details, but rather is to be defined by the appended claims.

Claims

1. A method for detecting an elephant flow by inspecting the operations of a machine that operates on a host computer, the method comprising: at the host computer: detecting an initiation of a new data flow on the machine; tracking an amount of data being transferred in the data flow, the amount of data being associated with a pack with a marking, the marking comprising information associated with statistics, the amount of data being compared to a threshold value; and when the amount of data transferred exceeds a threshold, specifying the data flow as an elephant data flow and informing a managed forwarding element (MFE) executing on the host computer to process the specified elephant data flow differently from other non-elephant flows, data packets of the data flow identified as the elephant flow being marked with the unique flow identifier and flow-specific statistics.
2. The method of claim 1, wherein the machine is a virtual machine (VM).
3. The method of claim 1, wherein the machine is a first machine, wherein the new data flow is an outbound data flow from the first machine to a second different machine, or an inbound data flow from the second different machine to the first machine.
4. The method of claim 1, wherein the detecting comprises detecting a network connection that is being opened on the machine.
5. The method of claim 4, wherein the detecting the network connection comprises performing a network introspection operation on the machine to intercept a socket call being made to open the network connection.
6. The method of claim 1, wherein the tracking comprises specifying, if the amount of data being transferred is greater than a threshold value, that the data flow is an elephant flow.
7. The method of claim 1 further comprising sending a message regarding a detected elephant flow to an agent, wherein the agent receives the message and processes each packet in the detected elephant flow differently from other non-detected flows.

8. The method of claim 7, wherein the machine is a source machine, wherein informing the MFE comprises providing contextual information to the MFE regarding the specified elephant flow.
9. The method of claim 8, wherein the context information includes at least one of the name of the source machine, an address associated with the source machine, an address associated with the destination machine, a port number, and an identification of an application that initiated the new data flow.
10. The method of claim 8, wherein the context information includes at least one of the name of the source machine, an address associated with the source machine, an address associated with the destination machine, a port number, and an identification of a user associated with the elephant flow.
11. A non-transitory machine readable medium storing a program that when executed by at least one processing unit of a host computer detects an elephant flow by inspecting the operations of a machine that operates on a host computer, the program comprising sets of instructions for: detecting an initiation of a new data flow on the machine; tracking an amount of data being transferred in the data flow, the amount of data being associated with a pack with a marking, the marking comprising information associated with statistics, the amount of data being compared to a threshold value; and when the amount of data transferred exceeds a threshold, specifying the data flow as an elephant data flow and informing a managed forwarding element (MFE) executing on the host computer to process the specified elephant data flow differently from other non-elephant flows, data packets of the data flow identified as the elephant flow being marked with the unique flow identifier and flow-specific statistics.
12. The non-transitory machine readable medium of claim 11, wherein the machine is a virtual machine (VM).
13. The non-transitory machine readable medium of claim 11, wherein the machine is a first machine, wherein the new data flow is an outbound data flow from the first machine to a second different machine, or an inbound data flow from the second different machine to the first machine.
14. The non-transitory machine readable medium of claim 11, wherein the set of instructions for detecting comprises a set of instructions for detecting a network connection that is being opened on the machine.
15. The non-transitory machine readable medium of claim 14, wherein the set of instructions for detecting the network connection comprises a set of instructions for performing a network introspection operation on the machine to intercept a socket call being made to open the network connection.
16. The non-transitory machine readable medium of claim 11, wherein the set of instructions for tracking comprises a set of instructions for specifying, if the amount of data being transferred is greater than a threshold value, that the data flow is an elephant flow.
17. The non-transitory machine readable medium of claim 11, the program further comprising a set of instructions for sending a message regarding a detected elephant flow to an agent, wherein the agent receives the message and processes each packet in the detected elephant flow different from other non-detected flows.
18. The non-transitory machine readable medium of claim 17, wherein the machine is a source machine, wherein the set of instructions for informing the MFE comprises a set of instructions for providing contextual information to the MFE regarding the specified elephant flow.
19. The non-transitory machine readable medium of claim 18, wherein the context information includes at least one of the name of the source machine, an address associated with the source machine, an address associated with the destination machine, a port number, and an identification of an application that initiated the new data flow.
20. The non-transitory machine readable medium of claim 18, wherein the context information includes at least one of the name of the source machine, an address associated with the source

machine, an address associated with the destination machine, a port number, and an identification of a user associated with the elephant flow.
